

Lab Assignment #11

Math 437 - Modern Data Analysis

Instructions

In this lab we will look at decision trees and ensembles of decision trees for both classification and regression.

```
here::i_am("Labs/Lab11.qmd")

# You may not need all of this, but you'll need most of it
library(ISLR2)
library(dplyr)
library(rpart)
library(randomForest)
library(gbm)
library(tidymodels)
library(ranger)
library(vip)
library(xgboost)

# load and split public_2020 dataset
public <- readr::read_csv(here::here("Data/public_2020.csv"))
public <- public |>
  mutate(
    MSI = if_else(HBCU | HSI, "Yes", "No") |> as.factor()
  )

set.seed(3745)
public_split <- initial_split(
  public,
  prop = 0.75
)
public_train <- training(public_split)
public_test <- testing(public_split)
```

This lab assignment is worth a total of **25 points** with up to 5 points of extra credit.

Problem 1: Decision Trees

In past years, students have complained that the `tree` package wasn't running on their version of R. So instead of doing Labs 8.3.1 and 8.3.2 we will run similar code using the `rpart` package.

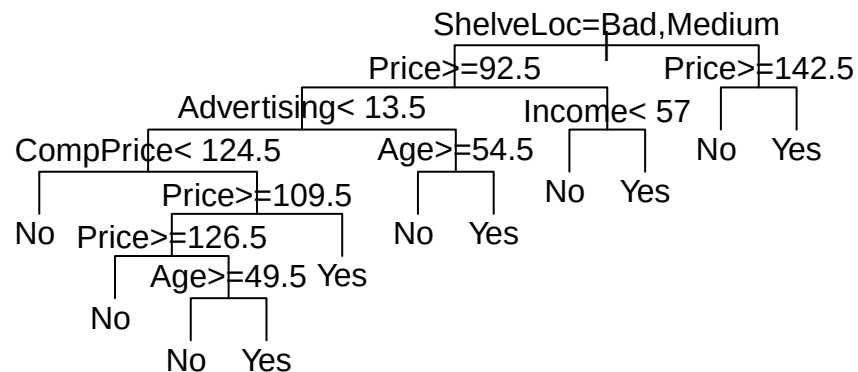
I have written each of the code chunks below to match Labs 8.3.1 and 8.3.2 as closely as possible. For each part, run the corresponding code chunks and answer the question about the output. You may find reading the text in Labs 8.3.1 and 8.3.2 to be useful.

Part a (Explanation: 1 pt)

```
Carseats2 <- Carseats |>
  mutate(
    High = if_else(Sales <= 8, "No", "Yes"))

tree.carseats <- rpart(High ~. - Sales, data = Carseats2)

par(xpd = TRUE)
plot(tree.carseats, uniform = TRUE)
text(tree.carseats, pretty = 0)
```



Briefly explain how to interpret this tree: which stores would end up in the very left “no” group, and which stores would end up in the very right “yes” group?

Stores that end up in the very left “no” group would be stores with bad shelf locations, prices that are less than 92.50 dollars, spend less than 13,500 dollars on advertising, and their Competitor price is less than 124.50 dollars. Stores that end up in the very right “yes” group would be stores with good shelf locations and prices that are 142.50 dollar or more.

Solution

Part b (Explanation: 1 pt)

```
tree.carseats
```

```
n= 400
```

```
node), split, n, loss, yval, (yprob)
    * denotes terminal node
```

```
1) root 400 164 No (0.59000000 0.41000000)
  2) ShelveLoc=Bad,Medium 315 98 No (0.68888889 0.31111111)
```

```

4) Price>=92.5 269 66 No (0.75464684 0.24535316)
8) Advertising< 13.5 224 41 No (0.81696429 0.18303571)
16) CompPrice< 124.5 96 6 No (0.93750000 0.06250000) *
17) CompPrice>=124.5 128 35 No (0.72656250 0.27343750)
34) Price>=109.5 107 20 No (0.81308411 0.18691589)
68) Price>=126.5 65 6 No (0.90769231 0.09230769) *
69) Price< 126.5 42 14 No (0.66666667 0.33333333)
138) Age>=49.5 22 2 No (0.90909091 0.09090909) *
139) Age< 49.5 20 8 Yes (0.40000000 0.60000000) *
35) Price< 109.5 21 6 Yes (0.28571429 0.71428571) *
9) Advertising>=13.5 45 20 Yes (0.44444444 0.55555556)
18) Age>=54.5 20 5 No (0.75000000 0.25000000) *
19) Age< 54.5 25 5 Yes (0.20000000 0.80000000) *
5) Price< 92.5 46 14 Yes (0.30434783 0.69565217)
10) Income< 57 10 3 No (0.70000000 0.30000000) *
11) Income>=57 36 7 Yes (0.19444444 0.80555556) *
3) ShelfLoc=Good 85 19 Yes (0.22352941 0.77647059)
6) Price>=142.5 12 3 No (0.75000000 0.25000000) *
7) Price< 142.5 73 10 Yes (0.13698630 0.86301370) *

```

What do the numbers in the parentheses mean? What do the stars mean?

Solution

The numbers in the parenthesis is probability of being no and yes in that order. Stars refer to terminal nodes, the end points of the tree

Part c (Explanation: 0.5 pts)

```

set.seed(2)
train <- sample(1:nrow(Carseats2), 200)
Carseats.test <- Carseats2[-train,]

tree.carseats <- rpart(High ~ . - Sales,
                        data = Carseats2,
                        subset = train,
                        control = rpart.control(xval = 10))
names(tree.carseats)

```

```
[1] "frame"           "where"           "call"
[4] "terms"           "cptable"         "method"
[7] "parms"           "control"         "functions"
[10] "numresp"         "splits"          "csplit"
[13] "variable.importance" "y"              "ordered"
```

```
tree.pred <- predict(tree.carseats, Carseats.test, type = "class")
table(tree.pred, Carseats.test$High, dnn = c("Predicted", "Actual"))
```

	Actual	
Predicted	No	Yes
No	97	25
Yes	20	58

What does the argument `xval = 10` mean?

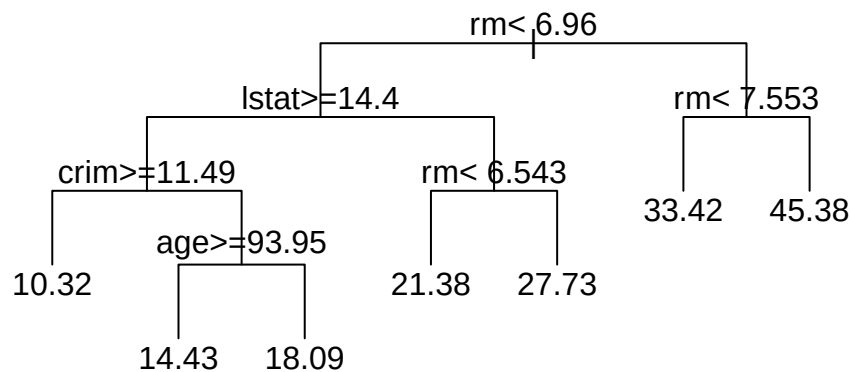
Solution

10 fold cross validation.

Part d (Explanation: 1 pt)

```
set.seed(1, kind = "Mersenne-Twister")
n <- nrow(Boston)
train <- sample(1:n, n/2)
tree.boston <- rpart(medv~., Boston, subset = train, control = rpart.control(xval = 10))
```

```
par(xpd = TRUE)
plot(tree.boston, uniform = TRUE)
text(tree.boston, pretty = 0)
```



Briefly explain how to interpret this tree: which neighborhoods would be predicted to have a `medv` of 10.32? Which neighborhoods would be predicted to have a `medv` of 27.73?

Solution

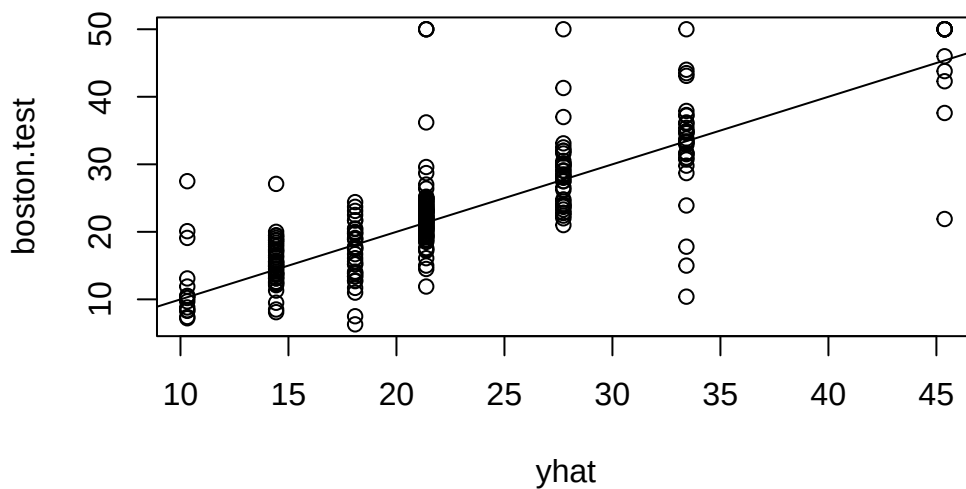
Neighborhoods with a `medv` of 10.32 have less than 6.96 average number of rooms, have 14.4% or greater lower status population, and have 11.49 or greater per capita crime in town. Neighborhoods with a `medv` of 27.73 have less than 6.96 average number of rooms, have less than 14.4% lower status population, and have 6.53 average number of rooms or more.

Part e (Explanation: 1 pt)

```

yhat <- predict(tree.boston, Boston[-train,])
boston.test <- Boston$medv[-train]
plot(yhat, boston.test)
abline(0,1)

```



```
mean((yhat - boston.test)^2)
```

```
[1] 35.28688
```

Why do we get this pattern - a bunch of vertical lines - in the plot? (HINT: We saw plots like this in the example code from lecture, but they were horizontal lines because the predictions were on the vertical axis instead.)

Solution

This is because regression trees aren't continuous, giving discrete type behavior due to piecewise constant predictions. With multiple observations sharing the same predicted values, vertical lines are created.

Part f (Code: 2.5 pts; Explanation: 1.5 pts)

We are now going to predict which public institutions are minority-serving institutions (HBCU or HSI).

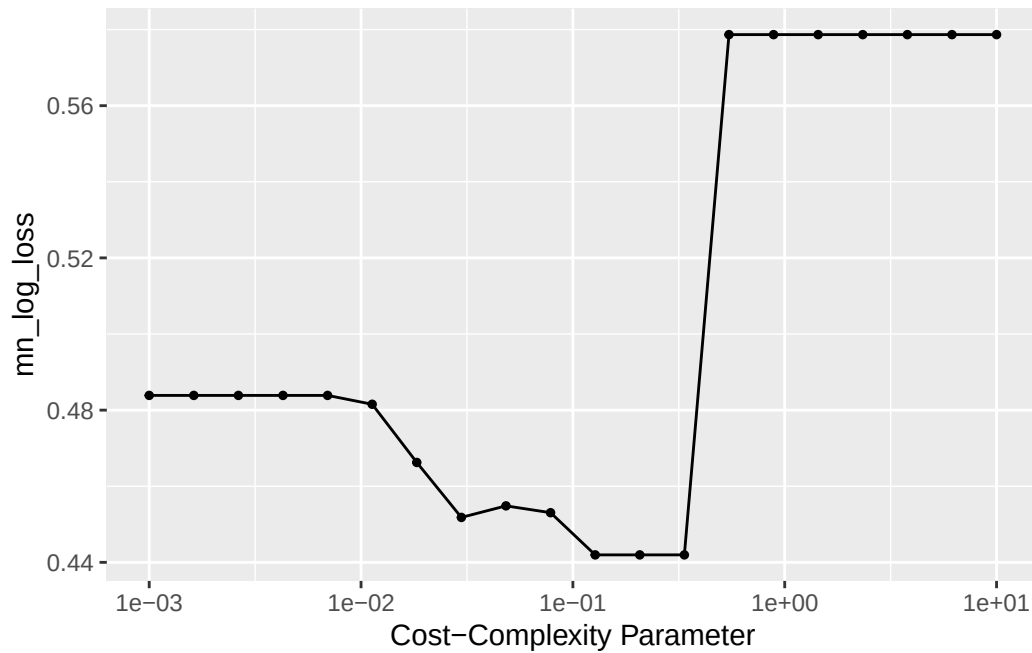
Using tidymodels with the `rpart` package, fit a classification tree model on the training set. Tune the cost-complexity parameter using a grid of values between 0.01 and 1 with a reasonable

enough number of levels so that you can actually see how your error metric of choice changes as a function of the cost-complexity parameter.

Choose a cost-complexity parameter by either Brier score or mean log-loss and fit that “best” model on the training set. Plot the final classification tree and explain how to use the tree to predict whether a school is MSI vs. non-MSI.

Solution

```
treeC_model <- decision_tree(  
  mode = "classification",      # classification or regression  
  engine = "rpart",            # use rpart package to fit the tree  
  cost_complexity = tune()      # tune this, but leave min_n and max_depth at default  
)  
  
treeC_recipe <- recipe(  
  MSI ~ .,  
  data = public_train  
) |>  
  step_rm(HBCU, HSI, Name)  
  
treeC_wflow <- workflow() |>  
  add_model(treeC_model) |>  
  add_recipe(treeC_recipe)  
  
set.seed(67)  
  
public_kfold <- vfold_cv(public_train, v = 5, repeats = 3)  
  
treeC_tune <- tune_grid(  
  treeC_wflow,  
  resamples = public_kfold,  
  metrics = metric_set(mn_log_loss),  
  grid = grid_regular(cost_complexity(range = c(-3, 1)), levels = 20))  
  
autoplot(treeC_tune)
```

```
treeC_best <- select_by_one_std_err(
  treeC_tune,
  metric = "mn_log_loss",
  desc(cost_complexity)
)
```

```
treeC_best
```

```
# A tibble: 1 x 2
  cost_complexity .config
      <dbl> <chr>
1      0.336 pre0_mod13_post0
```

```
treeC_wflow_final <- finalize_workflow(
  treeC_wflow,
  parameters = treeC_best
)

treeC_fit <- fit(treeC_wflow_final, data = public_train)

treeC_fit
```

```

== Workflow [trained] =====
Preprocessor: Recipe
Model: decision_tree()

-- Preprocessor -----
1 Recipe Step

* step_rm()

-- Model -----
n= 300

node), split, n, loss, yval, (yprob)
      * denotes terminal node

1) root 300 79 No (0.7366667 0.2633333)
   2) Pell_Grant< 0.56645 249 36 No (0.8554217 0.1445783) *
   3) Pell_Grant>=0.56645 51 8 Yes (0.1568627 0.8431373) *

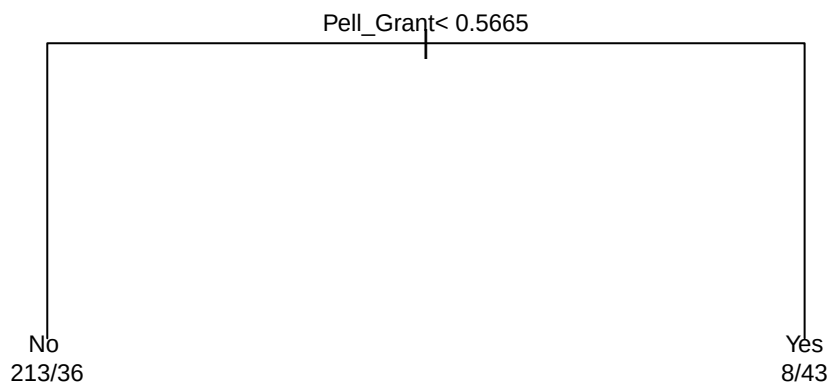
```

```

par(xpd = TRUE) # otherwise things clip
extract_fit_engine(treeC_fit) |>
  plot(compress = TRUE, margin = 0, uniform = TRUE)
# uniform = TRUE makes the tree drawing more uniform
# helps to space out the text issues

extract_fit_engine(treeC_fit) |>
  text(use.n = TRUE, minlength = 4, cex = 0.75)

```



```
# cex = 0.75 puts text at 3/4 size for readability
```

If Pell Grant is less than 0.5665, then the school is classified as non-MSI, “No”. If Pell Grant is at 0.5665 or higher, then the school is classified as MSI, “Yes”. The numbers at each end point show the counts of observations in each class, the prediction is based on the majority class.

Part g (Code: 1 pt)

Using your final model from Part f, make predictions on the test set. Obtain the confusion matrix and compute the estimated Brier score or mean log-loss.

Solution

```
treeC_predictions <- treeC_fit |>
  augment(new_data = public_test)

treeC_predictions
```

```
# A tibble: 101 x 21
  .pred_class .pred_No .pred_Yes Name      Location HBCU   HSI Admission_Rate
```

	<fct>	<dbl>	<dbl>	<chr>	<chr>	<dbl>	<dbl>	<dbl>
1	No	0.855	0.145	Universit~	Urban	0	0	0.771
2	No	0.855	0.145	The Unive~	Urban	0	0	0.804
3	Yes	0.157	0.843	Universit~	Urban	1	0	0.597
4	No	0.855	0.145	Universit~	Urban	0	0	0.938
5	No	0.855	0.145	Californi~	Suburban	0	0	0.384
6	No	0.855	0.145	Universit~	Suburban	0	0	0.464
7	No	0.855	0.145	San Diego~	Urban	0	1	0.367
8	No	0.855	0.145	San Franc~	Urban	0	1	0.841
9	No	0.855	0.145	Colorado ~	Suburban	0	0	0.550
10	No	0.855	0.145	Colorado ~	Urban	0	1	0.943

i 91 more rows

i 13 more variables: SAT <dbl>, Undergraduates <dbl>,
 # Average_Total_Cost <dbl>, Instructional_Expenditure <dbl>,
 # Faculty_Salary <dbl>, Full_Time_Faculty <dbl>, Pell_Grant <dbl>,
 # Retention_Rate <dbl>, Federal_Loan <dbl>, Default_Rate <dbl>,
 # Proportion_Men <dbl>, Endowment <dbl>, MSI <fct>

```
conf_mat(
  treeC_predictions,
  truth = MSI,
  estimate = .pred_class
)
```

	Truth	
Prediction	No	Yes
No	76	15
Yes	4	6

```
brier_class(treeC_predictions, truth = MSI, .pred_Yes)
```

```
# A tibble: 1 x 3
  .metric      .estimator .estimate
  <chr>        <chr>        <dbl>
1 brier_class binary          0.597
```

Problem 2: Random Forests and Boosted Trees

Part a (Code: 2 pts)

Run the code in ISLR Labs 8.3.3 and 8.3.4. Put each chunk from the textbook in its own chunk.

Solution

```
set.seed(1)

bag.boston <- randomForest(medv ~ .,
                           data = Boston,
                           subset = train, mtry = 12, importance = TRUE)

bag.boston
```

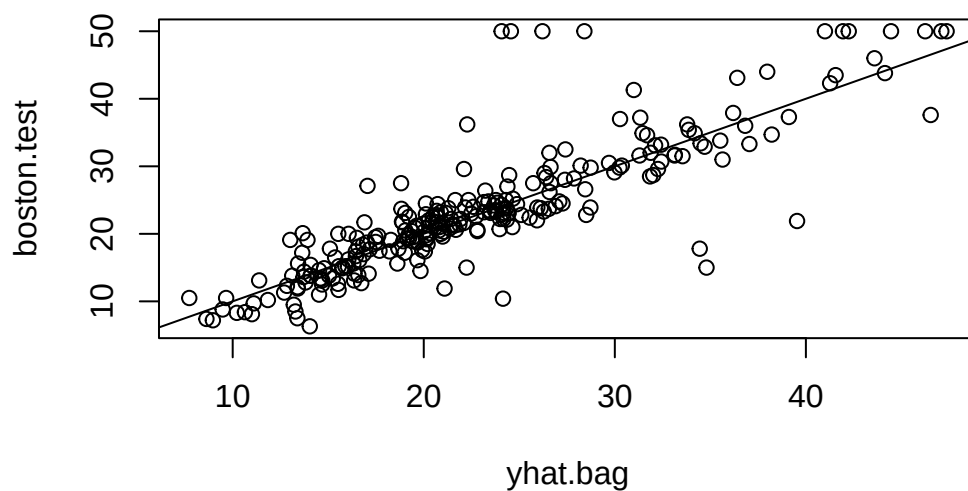
Call:

```
randomForest(formula = medv ~ ., data = Boston, mtry = 12, importance = TRUE, subset =
              Type of random forest: regression
              Number of trees: 500
No. of variables tried at each split: 12

              Mean of squared residuals: 11.40162
              % Var explained: 85.17
```

```
yhat.bag <- predict(bag.boston, newdata = Boston[-train, ])

plot(yhat.bag, boston.test)
abline(0, 1)
```



```
mean((yhat.bag - boston.test)^2)
```

```
[1] 23.41916
```

```
bag.boston <- randomForest(medv ~ .,
                           data = Boston,
                           subset = train, mtry = 12, ntree = 25)

yhat.bag <- predict(bag.boston, newdata = Boston[-train, ])

mean((yhat.bag - boston.test)^2)
```

```
[1] 25.75055
```

```
set.seed(1)

rf.boston <- randomForest(medv ~ .,
                          data = Boston,
                          subset = train, mtry = 6, importance = TRUE)

yhat.rf <- predict(rf.boston, newdata = Boston[-train, ])

mean((yhat.rf - boston.test)^2)
```

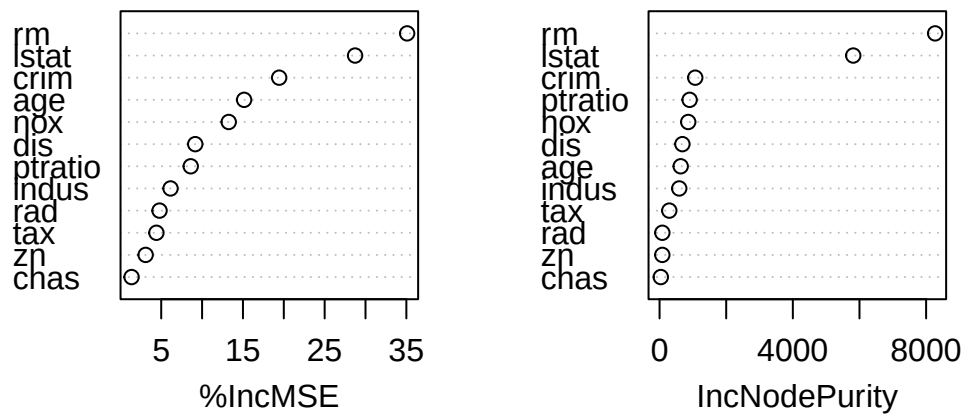
```
[1] 20.06644
```

```
randomForest::importance(rf.boston)
```

	%IncMSE	IncNodePurity
crim	19.435587	1070.42307
zn	3.091630	82.19257
indus	6.140529	590.09536
chas	1.370310	36.70356
nox	13.263466	859.97091
rm	35.094741	8270.33906
age	15.144821	634.31220
dis	9.163776	684.87953
rad	4.793720	83.18719
tax	4.410714	292.20949
ptratio	8.612780	902.20190
lstat	28.725343	5813.04833

```
varImpPlot(rf.boston)
```

rf.boston



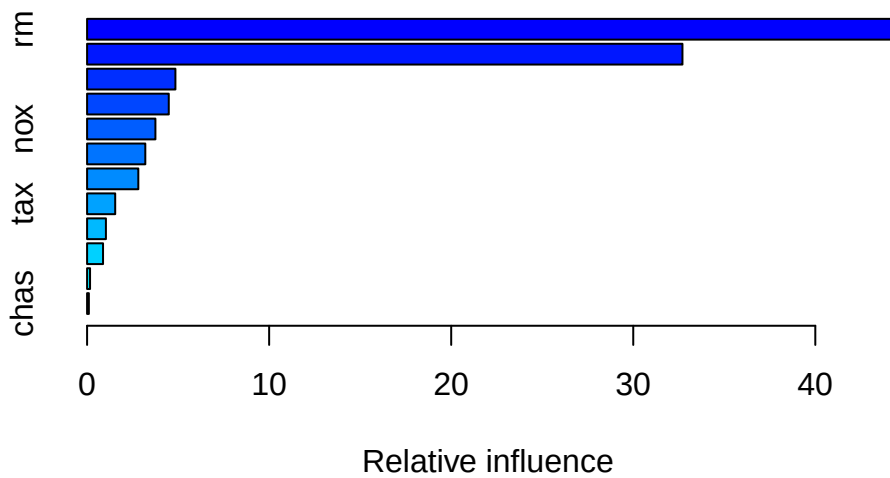
```

set.seed(1)

boost.boston <- gbm(medv ~ .,
                    data = Boston[train, ],
                    distribution = "gaussian", n.trees = 5000,
                    interaction.depth = 4)

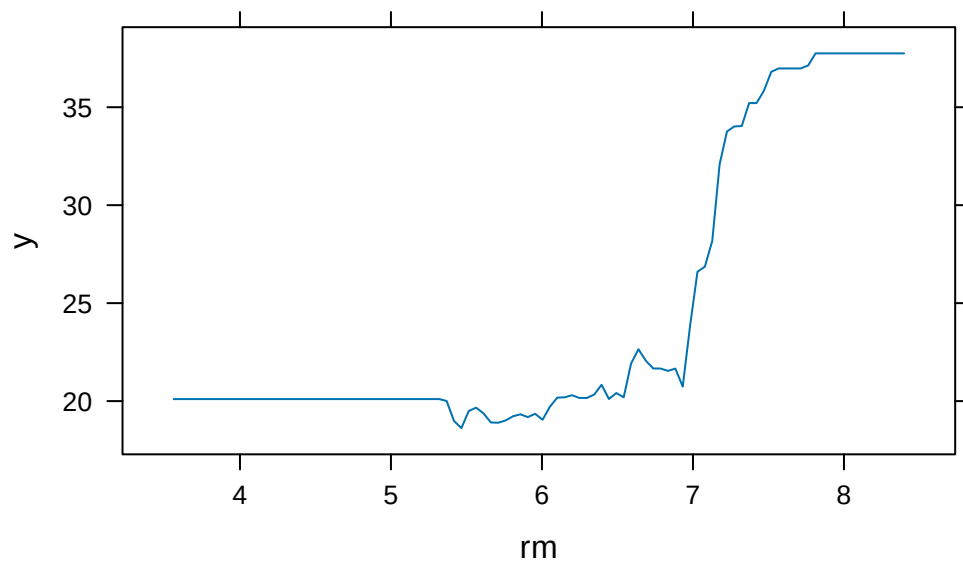
summary(boost.boston)

```

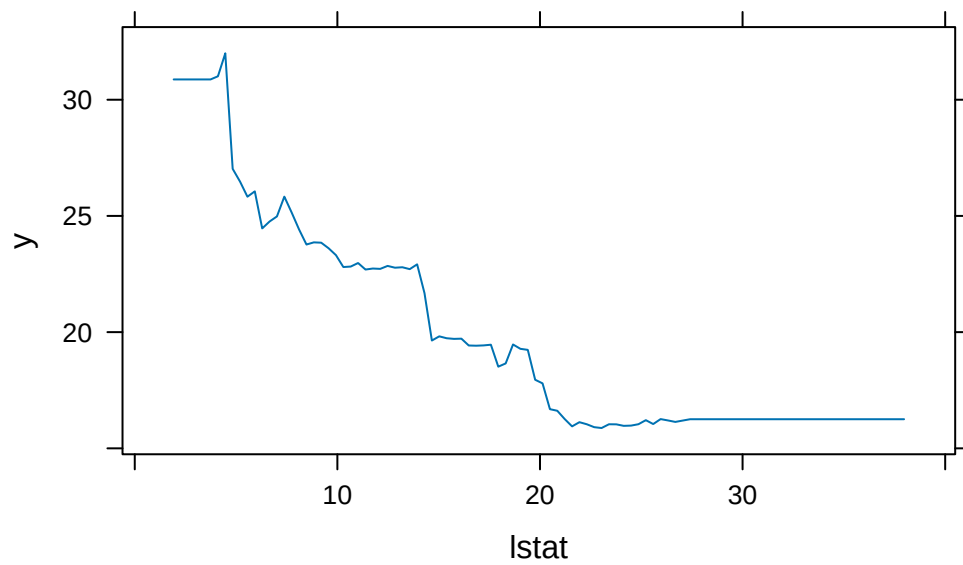


	var	rel.inf
rm	rm	44.48249588
lstat	lstat	32.70281223
crim	crim	4.85109954
dis	dis	4.48693083
nox	nox	3.75222394
age	age	3.19769210
ptratio	ptratio	2.81354826
tax	tax	1.54417603
indus	indus	1.03384666
rad	rad	0.87625748
zn	zn	0.16220479
chas	chas	0.09671228


```
plot(boost.boston, i = "rm")
```



```
plot(boost.boston, i = "lstat")
```



```

yhat.boost <- predict(boost.boston,
                      newdata = Boston[-train, ],
                      n.trees = 5000)

mean((yhat.boost - boston.test)^2)

```

```
[1] 18.39057
```

```

boost.boston <- gbm(medv ~ .,
                   data = Boston[train, ],
                   distribution = "gaussian",
                   n.trees = 5000,
                   interaction.depth = 4,
                   shrinkage = 0.2,
                   verbose = F)

yhat.boost <- predict(boost.boston,
                      newdata = Boston[-train, ],
                      n.trees = 5000)

mean((yhat.boost - boston.test)^2)

```

```
[1] 16.54778
```

Part b (Code: 2.5 pts)

Using tidymodels, create a random forest model predicting whether an institution is a minority-serving institution (MSI) from the other variables in the dataset (except, obviously, HBCU and HSI, and Name). The `rand_forest` function will accept either `randomForest` or `ranger` as the engine.

Make sure to use `set_args` to tell tidymodels you want to use permutation-based importance (you will need it for part d). Tune the value of `mtry` using cross-validation; use values of `mtry` from 1 to 14 (14 = bagging).

Select your best model by cross-validated Brier score and justify your choice. Using your best model, obtain the out-of-bag prediction error (Brier score) on the training set.

Solution

```

rfC_model <- rand_forest(
  mode = "classification",
  engine = "ranger"
) |>
  set_args(
    seed = 121,      # set INSIDE set_args
    mtry = tune(),
    importance = "permutation"
  )

rfC_recipe <- recipe(
  MSI ~ .,
  data = public_train
) |>
  step_rm(HBCU, HSI, Name)

rfC_wflow <- workflow() |>
  add_model(rfC_model) |>
  add_recipe(rfC_recipe)

rfC_grid <- expand_grid(mtry = seq(1, 14))

rfC_tune <- tune_grid(rfC_wflow,
  resamples = public_kfold,
  metrics = metric_set(brier_class),
  grid = rfC_grid)

rfC_best <- select_best(rfC_tune, metric = "brier_class")

rfC_best

```

```

# A tibble: 1 x 2
  mtry .config
  <int> <chr>
1     3 pre0_mod03_post0

```

```

rfC_final_wflow <- finalize_workflow(
  rfC_wflow,
  parameters = rfC_best
)

```

```
rfC_fit <- fit(rfC_final_wflow, data = public_train)
```

```
rfC_fit
```

```
== Workflow [trained] =====
```

```
Preprocessor: Recipe
```

```
Model: rand_forest()
```

```
-- Preprocessor -----
```

```
1 Recipe Step
```

```
* step_rm()
```

```
-- Model -----
```

```
Ranger result
```

```
Call:
```

```
  ranger::ranger(x = maybe_data_frame(x), y = y, mtry = min_cols(~3L, x), seed = ~121, in
```

```
Type: Probability estimation
```

```
Number of trees: 500
```

```
Sample size: 300
```

```
Number of independent variables: 14
```

```
Mtry: 3
```

```
Target node size: 10
```

```
Variable importance mode: permutation
```

```
Splitrule: gini
```

```
OOB prediction error (Brier s.): 0.1130641
```

```
rfC_engine <- rfC_fit |> extract_fit_engine()
```

```
rfC_engine |> pluck("prediction.error")
```

```
[1] 0.1130641
```

Part c (Code: 1 pt; Explanation: 0.5 pts)

Using the model chosen in Part b, obtain predictions on the test set and compute the estimated test Brier score. Compare Brier score on the test set to the cross-validated mean Brier score and out-of-bag prediction error on the training set.

Solution

```
rfC_predictions <- rfC_fit |>
  augment(new_data = public_test)
```

```
rfC_predictions
```

```
# A tibble: 101 x 21
```

	.pred_class	.pred_No	.pred_Yes	Name	Location	HBCU	HSI	Admission_Rate
	<fct>	<dbl>	<dbl>	<chr>	<chr>	<dbl>	<dbl>	<dbl>
1	No	0.938	0.0616	Universit~	Urban	0	0	0.771
2	No	0.911	0.0893	The Unive~	Urban	0	0	0.804
3	Yes	0.114	0.886	Universit~	Urban	1	0	0.597
4	No	0.969	0.0307	Universit~	Urban	0	0	0.938
5	No	0.855	0.145	Californi~	Suburban	0	0	0.384
6	No	0.506	0.494	Universit~	Suburban	0	0	0.464
7	No	0.579	0.421	San Diego~	Urban	0	1	0.367
8	Yes	0.460	0.540	San Franc~	Urban	0	1	0.841
9	No	0.843	0.157	Colorado ~	Suburban	0	0	0.550
10	No	0.769	0.231	Colorado ~	Urban	0	1	0.943

```
# i 91 more rows
```

```
# i 13 more variables: SAT <dbl>, Undergraduates <dbl>,
```

```
# Average_Total_Cost <dbl>, Instructional_Expenditure <dbl>,
```

```
# Faculty_Salary <dbl>, Full_Time_Faculty <dbl>, Pell_Grant <dbl>,
```

```
# Retention_Rate <dbl>, Federal_Loan <dbl>, Default_Rate <dbl>,
```

```
# Proportion_Men <dbl>, Endowment <dbl>, MSI <fct>
```

```
conf_mat(
  rfC_predictions,
  truth = MSI,
  estimate = .pred_class
)
```

	Truth	
Prediction	No	Yes
No	75	10
Yes	5	11

```
show_best(rfC_tune, metric = "brier_class") # CV Brier
```

```
# A tibble: 5 x 7
  mtry .metric      .estimator mean      n std_err .config
  <int> <chr>         <chr>    <dbl> <int>   <dbl> <chr>
1     3 brier_class binary    0.116   15 0.00419 pre0_mod03_post0
2     2 brier_class binary    0.116   15 0.00409 pre0_mod02_post0
3     4 brier_class binary    0.117   15 0.00452 pre0_mod04_post0
4     5 brier_class binary    0.119   15 0.00454 pre0_mod05_post0
5     1 brier_class binary    0.120   15 0.00381 pre0_mod01_post0
```

```
rfC_engine |> pluck("prediction.error") # OOB Brier
```

```
[1] 0.1130641
```

```
brier_class(rfC_predictions, truth = MSI, .pred_No) # Test Brier
```

```
# A tibble: 1 x 3
  .metric      .estimator .estimate
  <chr>         <chr>         <dbl>
1 brier_class binary    0.110
```

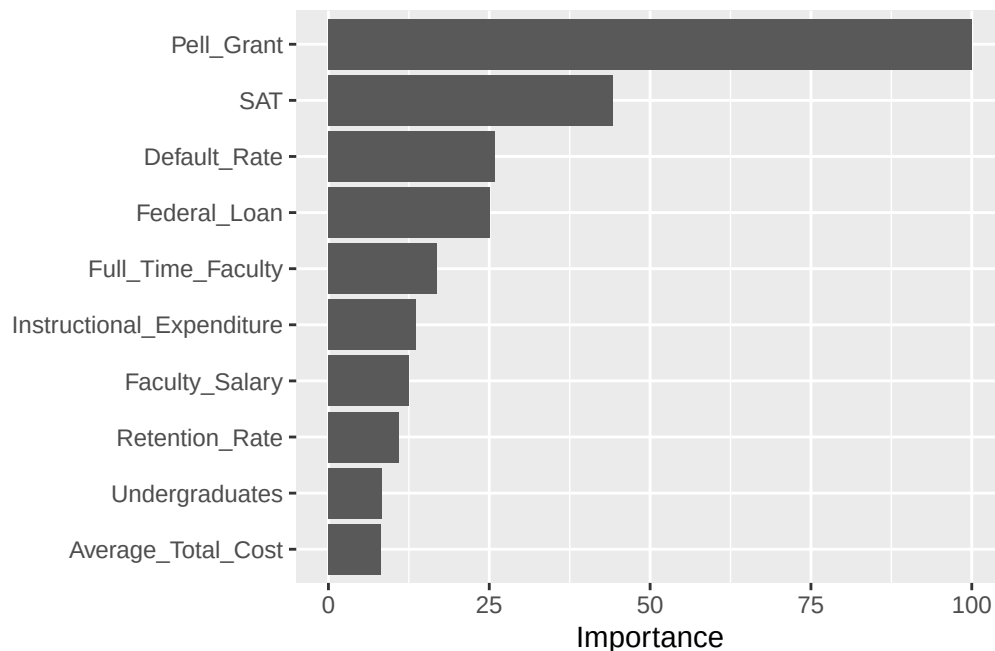
To compare, Brier score just got better as we went. The cross-validated mean Brier score was .1161. The out-of-bag prediction error on the training set was .1131. Finally, the estimated test Brier score was .1099.

Part d (Code: 0.5 pts; Explanation: 1 pt)

Obtain and plot estimates of variable importance (%IncMSE column if you are using `randomForest` or `variable.importance` if you are using `ranger`) for the model chosen in Part b. Which predictor variables have the largest importance and are thus the most useful for making the predictions?

Solution

```
rfC_fit |> extract_fit_engine() |>
  vip(scale = TRUE)
```



The most important predictor to determine if some university is a Minority Serving Institution is the Pell Grant, so in the graph, all else is scaled to that predictor. Aside from the main one, the top 3 important predictors are SAT scores, Default Rate, and Federal Loan, with the SAT scores definitely overshadowing the latter 2.

Part e (Code: 2.5 pts)

Using `tidymodels` with the `xgboost` package, fit a gradient-boosted tree model on the training set using the same predictors as the decision tree and random forest models. Tune only the 3 hyperparameters discussed in lecture: the number of trees to fit (`trees`), interaction depth (`tree_depth`), and learning rate (`learn_rate`). In order for this to not take forever, search over a small grid: 50, 100, 250, and 500 for `trees`, 1 and 2 for `tree_depth`, and 0.01 and 0.10 for `learn_rate`.

Select your best model by either Brier score or mean log-loss and justify your choice.

Solution

```
xgboostC_model <- boost_tree(
  mode = "classification",
  engine = "xgboost",
  trees = tune(),
```

```

    tree_depth = tune(),
    learn_rate = tune()
  )

  xgboostC_recipe <- recipe(
    MSI ~ .,
    data = public_train
  ) |>
    step_rm(HSI, HBCU, Name) |>
    step_dummy(all_nominal_predictors())

  xgboostC_wflow <- workflow() |>
    add_model(xgboostC_model) |>
    add_recipe(xgboostC_recipe)

  set.seed(67)

  xgboostC_tune <- tune_grid(xgboostC_wflow,
    resamples = public_kfold,
    metrics = metric_set(brier_class, mn_log_loss),
    grid = expand_grid(
      trees = c(50, 100, 250, 500),
      tree_depth = c(1,2), # must specify lower and upper bounds, here we c
      learn_rate = c(.01, .1))
    )

  xgboostC_best <- select_best(xgboostC_tune, metric = "brier_class")

  xgboostC_best

```

```

# A tibble: 1 x 4
  trees tree_depth learn_rate .config
  <dbl>     <dbl>     <dbl> <chr>
1   250         1         0.1 pre0_mod10_post0

```

```

xgboostC_wflow_final <- finalize_workflow(xgboostC_wflow, parameters = xgboostC_best)

xgboostC_fit <- fit(xgboostC_wflow_final, data = public_train)

xgboostC_fit

```

```

== Workflow [trained] =====

```



```
Preprocessor: Recipe
Model: boost_tree()
```

```
-- Preprocessor -----
2 Recipe Steps

* step_rm()
* step_dummy()

-- Model -----
##### xgb.Booster
call:
  xgboost::xgb.train(params = list(eta = 0.1, max_depth = 1, gamma = 0,
    colsample_bytree = 1, colsample_bynode = 1, min_child_weight = 1,
    subsample = 1, nthread = 1, objective = "binary:logistic"),
    data = x$data, nrounds = 250, evals = x$watchlist, verbose = 0)
# of features: 16
# of rounds: 250
callbacks:
  evaluation_log
evaluation_log:
  iter training_logloss
  <num>          <num>
    1          0.5455152
    2          0.5221530
  ---          ---
  249          0.2103418
  250          0.2100237
```

Part f (Code: 1 pt)

Using your best model from Part e, make predictions on the test set. Obtain the confusion matrix and the estimated Brier score or mean log-loss on the test set.

Solution

```
xgboostC_predictions <- augment(xgboostC_fit, new_data = public_test)

xgboostC_predictions
```

```
# A tibble: 101 x 21
```

	.pred_class	.pred_No	.pred_Yes	Name	Location	HBCU	HSI	Admission_Rate
	<fct>	<dbl>	<dbl>	<chr>	<chr>	<dbl>	<dbl>	<dbl>
1	No	0.906	0.0945	Universit~	Urban	0	0	0.771
2	No	0.961	0.0391	The Unive~	Urban	0	0	0.804
3	Yes	0.0416	0.958	Universit~	Urban	1	0	0.597
4	No	0.957	0.0426	Universit~	Urban	0	0	0.938
5	No	0.955	0.0454	Californi~	Suburban	0	0	0.384
6	No	0.516	0.484	Universit~	Suburban	0	0	0.464
7	No	0.675	0.325	San Diego~	Urban	0	1	0.367
8	Yes	0.254	0.746	San Franc~	Urban	0	1	0.841
9	No	0.985	0.0153	Colorado ~	Suburban	0	0	0.550
10	No	0.897	0.103	Colorado ~	Urban	0	1	0.943

i 91 more rows

i 13 more variables: SAT <dbl>, Undergraduates <dbl>,

Average_Total_Cost <dbl>, Instructional_Expenditure <dbl>,

Faculty_Salary <dbl>, Full_Time_Faculty <dbl>, Pell_Grant <dbl>,

Retention_Rate <dbl>, Federal_Loan <dbl>, Default_Rate <dbl>,

Proportion_Men <dbl>, Endowment <dbl>, MSI <fct>

```
conf_mat(
  xgboostC_predictions,
  truth = MSI,
  estimate = .pred_class
)
```

	Truth	
Prediction	No	Yes
No	76	13
Yes	4	8

```
brier_class(xgboostC_predictions, truth = MSI, .pred_No)
```

A tibble: 1 x 3

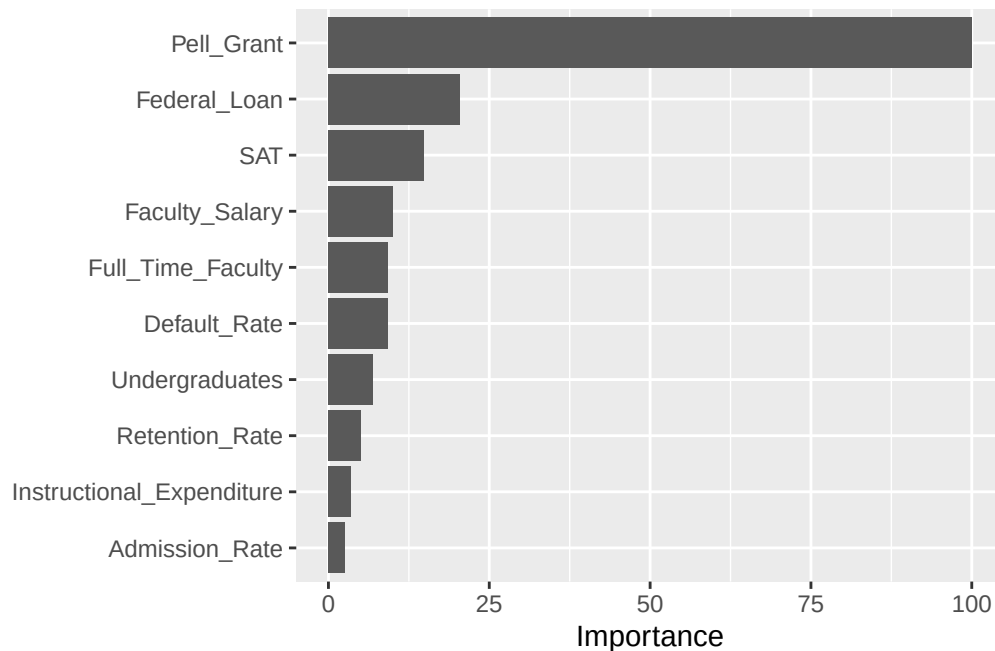
	.metric	.estimator	.estimate
	<chr>	<chr>	<dbl>
1	brier_class	binary	0.116

Part g (Code: 0.5 pts; Explanation: 1 pt)

Produce a variable importance plot for your xgboost model from Part e. Compare the (relative) variable importances between the random forest and xgboost models.

Solution

```
xgboostC_fit |> extract_fit_engine() |>  
  vip(scale = TRUE)
```



In this VIP chart, Pell Grant remains the most important, following right below it at 25% is Federal Loan. SAT scores takes another jump to a value close to 13%, and then faculty and so on and so forth. So, I would say the main difference between the two is that SAT and Federal Loan swaps with each other on each graph for 2nd and 3rd in terms of importance, however, SAT carries much more relative importance in the Random Forest VIP graph than the Federal Loan does in the xgboost VIP graph. This information tells us this about the model itself and not just some graphs. After that a lot of variables switch places. Another main point is that variables carry more importance overall in general in the Random Forest model than they do in the xgboost model.

Part h (Explanation: 1 pt)

Consider the following statement: the second decision tree that is fit does not depend on the first decision tree that is fit. Is that statement TRUE or FALSE for the random forest algorithm? What about for boosted trees? Explain your reasoning.

Solution

The statement is true for random forests, each tree is done separately with a bootstrap sample of the data and a random subset of predictors, so, the second tree has nothing to with the first tree in any way. but FALSE for boosted trees.

The statement is false for boosted trees. Boosting builds trees off its prior tree. Each new tree is fit to the residuals of the previous trees. Therefore, the second tree clearly depends on the first tree.

Problem 3: Neural Networks

Part a (Computation and Explanation: 2 pts)

Consider two predictors, x_1 and x_2 , feeding into a two-unit hidden layer. Suppose that the hidden layer uses ReLU activation functions with w_{kj} given in Equation (10.6) of the book and the output layer uses a linear activation function with β_j given in equation (10.6).

Find the activation functions $A_1(x_1, x_2)$ and $A_2(x_1, x_2)$ and $f(x_1, x_2)$. Note: $f(x_1, x_2)$ should be a piecewise function.

Solution

```
x <- "hello world"
print(x)
```

```
[1] "hello world"
```

Part b (Computation: 5 pts Extra Credit)

No.